

CivicSwarm

50 complaints about the same burst pipe → 1 escalated case, 1 field crew

Theme: Theme 05 — Agent Swarms · Function: AI-Powered Public Services

Suggested stack: Node.js 22 · Groq (Llama 3.1) · SQLite · React 18 + React Flow · WebSocket

Problem Statement

Problem Background

Cities receive thousands of complaints daily — potholes, water leaks, broken lights, sanitation failures — across phone lines, web portals, and apps. Each must be classified, routed to the right department, assessed, and resolved. Done manually, the process is slow: complaints queue for days, get mis-routed to the wrong crew, and the same underlying issue gets treated as dozens of unrelated tickets. When a water main bursts on Pine Street, fifty residents file fifty individual complaints. Without intelligent clustering, that generates fifty work orders, fifty citizen responses, and five separate crew dispatches to the same street — while the root cause goes unaddressed for hours.

Why It Matters

Responsive local government is the most visible form of public trust. Resolving a complaint in hours builds confidence; losing it in a backlog erodes it. The bottleneck is not intake — it is the parallel attention and pattern recognition required to classify complaints accurately, assess them with domain knowledge, detect when many reports share one root cause, and draft the right response. A swarm of specialist agents running concurrently inverts this asymmetry.

Solution Summary

Why This Problem Was Chosen

Grievance resolution is naturally parallel and clustering-driven: departments need different handling, and the highest-value insight — many complaints are one problem — emerges only when a swarm clusters across the whole intake. The root-cause clustering capability is something no single-agent system can do reliably.

Proposed Solution

As complaints arrive, an Orchestrator fans them out to four specialist agents: Intake classifies and structures raw complaints, Clustering detects when many complaints share a root cause and collapses them into one Case, Department applies domain-specialist

assessment (water, roads, sanitation, lighting) including crew and equipment requirements, and Resolution drafts citizen responses and work orders. When clustering identifies a systemic issue, the Orchestrator creates one escalated Case from many tickets — one coordinated work order, one consistent message to all affected residents. Officers review and approve all AI-drafted outputs via a React console before anything goes out.

Expected Impact

- Clear complaint backlogs faster via parallel, auto-routed processing
- Collapse duplicates into single root-cause cases — fix the pipe once, not fifty times
- Domain-aware routing and assessment reduces mis-routing and false closures
- Officers stay in control — the swarm drafts, humans approve

Technical Approach & Implementation

Solution Workflow

- Complaint batch arrives via REST API (or mock batch buttons in demo)
- Orchestrator creates a 4-node resolution DAG and persists batch to SQLite
- Phase 1 (parallel): Intake classifies all complaints + Clustering scans ALL stored complaints simultaneously
- Clustering detects root-cause groups: same ward + related type + within time window + count $\geq$ threshold
- Phase 2: Department agent assesses each complaint with domain knowledge — crew, equipment, SOP, hours
- Phase 3: Resolution agent drafts citizen response and work order (individual or escalated)
- Officer reviews AI drafts in React console and clicks Approve or Reject

Key Features

- Root-cause clustering — detects when 50 complaints are 1 burst pipe and collapses to 1 escalated Case
- Parallel resolution flow — Intake and Clustering run simultaneously, not sequentially
- Department-specialist assessment — domain knowledge for water, roads, sanitation, lighting
- Human-approved automation — all drafts require officer approval before any action is taken
- React Flow pipeline visualisation — live 4-node DAG showing agent status in real time

Technology Stack

Frontend: React 18 + plain JS, React Flow (pipeline DAG), plain React cards (cluster map), WebSocket

Backend: Node.js 22 + Express, Groq (Llama 3.1) for intake/department/resolution, WebSocket (ws)

AI/ML: Llama 3.1-8b-instant via Groq, mock fallback for offline dev. Clustering is

deterministic algorithm — no LLM

Data: SQLite with WAL mode (complaints, cases, batches, dags, findings) — zero infrastructure

Models & Algorithms

Clustering algorithm: groups complaints by (ward + normalised type) within TIME_WINDOW_HOURS. Related type pairs (water_leak ↔ flooding) are treated as equivalent. Groups with count $\geq$ CLUSTER_THRESHOLD (3) become Cases. Deterministic — same input always produces the same grouping. Intake uses a batched LLM call for all complaints in a single round-trip. Department prompt encodes SOP knowledge per complaint type. Resolution distinguishes individual vs escalated output based on case_id presence.

Innovation

- Root-cause clustering at scale — 50 complaints become 1 case, 1 work order, 1 coordinated fix
- Department-specialist resolution — domain knowledge applied at the right complaint type
- Parallel Intake + Clustering — both run simultaneously since they read independent data sources
- Human-approved automation — accountability stays with people, speed comes from the swarm

Future Scope

Near-term

- Multi-language and voice intake replacing text-only complaint submission
- GIS map of complaint clusters for field crew routing
- SLA tracking per department with breach alerts

Medium-term

- Investigation sub-agents for ambiguous complaints requiring record checks
- Cross-ward pattern reuse — solutions to recurring issues pre-populated
- Checkpoints before escalating systemic cases

Long-term

- Predictive maintenance from complaint pattern trends
- Cross-department federated swarms for multi-agency issues
- Continuous civic-health monitoring with proactive proposals

Scalability & Larger Vision

How It Scales

The pipeline is event-driven and stateless: complaint batches POST to the API, and agents run as async Node.js tasks. WAL-mode SQLite handles the parallel Phase 1 writes (Intake and Clustering write findings simultaneously without conflicts). The Clustering algorithm runs over ALL stored complaints — as the complaint history grows, the algorithm finds more root-cause patterns. Replacing SQLite with PostgreSQL for production scale is a one-file change to the storage layer.

How It Expands

Near term: GIS mapping and multi-language intake. Medium term: investigation sub-agents for complex complaints and cross-ward pattern reuse. Long term: predictive maintenance from complaint trends and continuous civic-health monitoring.

The Larger Vision

Municipal complaint handling stops being a reactive backlog and becomes a proactive service. The highest-value insight — many complaints are one systemic problem — is surfaced automatically every time, regardless of how many tickets come in. Officers spend time approving well-researched responses, not triaging noise.

Potential Impact

For a mid-sized city receiving 500 complaints per day, CivicSwarm reduces duplicate work orders by detecting root causes that span multiple tickets. At scale, the compounding effect is significant: faster resolution, fewer repeat complaints for the same underlying issue, and an officer team that spends its time on decisions rather than classification and routing.